

Operating Systems: The Interview Manual

A professor's rigor paired with a recruiter's ear. Seven topic areas — from first-principles fundamentals through production infrastructure — each question tagged Basic, Intermediate, or Advanced, with worked numerical examples, comparison tables, and real engineering incidents sourced from GATE syllabus archives, GeeksforGeeks, InterviewBit, LeetCode Discuss, Glassdoor reports, Linux kernel documentation, and postmortems published by Cloudflare and Uber Engineering.

79 questions

7 topic areas

Basic → Intermediate → Advanced

Worked examples: scheduling • Banker's algorithm • paging • disk arm

Sourced from GATE, GfG, LeetCode Discuss, Glassdoor, Linux kernel docs, Cloudflare & Uber Engineering

Updated Sep 2026

01 Core OS Fundamentals & Architectures

The questions that establish whether a candidate understands what an operating system actually *is* — the hardware/software boundary, the dual-mode protection model, and the vocabulary (traps, interrupts, kernels) that every later topic assumes you already have.

1.1 BASIC What is an operating system, and what are its core responsibilities?

An operating system is the resident software layer that sits between the raw hardware and every application program, acting simultaneously as a **resource manager** and an **extended (virtual) machine**. As a resource manager, it multiplexes finite physical resources — CPU time, RAM, disk, and I/O devices — across competing processes fairly and safely. As a virtual machine, it hides messy hardware details (interrupt lines, disk geometry, device registers) behind clean abstractions: processes instead of raw CPU cycles, files instead of raw disk blocks, virtual address spaces instead of physical RAM.

Its five canonical responsibilities are **process management** (creation, scheduling, termination, IPC), **memory management** (allocation, protection, virtual memory), **file system management** (naming, storage, access control), **device/I/O management** (drivers, buffering, spooling), and **protection & security** (isolating processes and users from one another).

resource manager

virtual machine abstraction

multiplexing

1.2 BASIC

What is the difference between user mode and kernel mode, and why does this separation exist?

Modern CPUs implement at least two privilege rings, exposed by the OS as **user mode** and **kernel mode**. A mode bit in a protected CPU register tracks which one is active. In user mode, code cannot directly execute privileged instructions (halting the CPU, reprogramming the MMU, touching device registers) or access arbitrary physical memory — any attempt raises a hardware exception. In kernel mode, the executing code has unrestricted access to the entire machine.

The separation exists for **fault isolation** and **security**: a bug or malicious instruction in a user application can only corrupt that process's own address space, so the OS kills the offending process and the rest of the system survives. If everything ran in one privilege level, one misbehaving pointer dereference could corrupt the disk driver or another user's memory. The only door between the two modes is a controlled one — the **system call** — never a direct jump.

dual-mode operation

mode bit

privilege ring

fault isolation

WHAT INTERVIEWERS LISTEN FOR

Say "mode bit" and "trap instruction" explicitly — vague answers that only say "kernel mode is more powerful" without naming the hardware mechanism read as memorized rather than understood.

1.3 BASIC

What is a system call, and what actually happens when a user program invokes one?

A system call is the programmatic interface through which a user-mode process requests a privileged service from the kernel — opening a file, forking a process, allocating memory, sending a network packet. The sequence: (1) the application calls a thin wrapper in the C library (e.g. `read()`), which loads a system-call number and arguments into registers; (2) it executes a special **trap instruction** (e.g. `syscall` on x86-64, `svc` on ARM) — a deliberate, controlled software interrupt; (3) the CPU hardware flips the mode bit to kernel mode and jumps to a fixed entry point in the kernel; (4) the kernel's syscall dispatcher validates the arguments and indexes into a syscall table to run the requested handler; (5) on completion, the kernel restores the saved user-mode context and executes a return-from-trap instruction, flipping the mode bit back and resuming the application exactly where it left off.

trap instruction

syscall table

libc wrapper

1.4 BASIC What's the difference between a system call, an interrupt, and a trap?

All three transfer control to the kernel, but by different triggers. An **interrupt** is asynchronous and externally generated by hardware — a disk finishing a transfer, a timer firing, a key press — and can occur between any two instructions regardless of what the CPU is doing. A **trap** (or exception) is synchronous and caused by the currently executing instruction itself, such as division by zero, an invalid memory access (page fault), or an illegal opcode. A **system call** is a deliberate, well-behaved trap — the program intentionally executes a trap instruction to request kernel service, as opposed to an accidental fault. In short: interrupts come from outside, traps come from within, and a system call is a voluntary trap.

synchronous vs asynchronous

exception

1.5 INTERMEDIATE Compare monolithic kernels, microkernels, and hybrid kernels.

- **Monolithic kernel** (Linux, traditional Unix): the entire OS — scheduler, memory manager, file systems, device drivers — runs as one large program in kernel mode/address space. Fast (all calls are simple function calls, no message passing) but a bug in any driver can crash the whole kernel.
- **Microkernel** (Minix, QNX, L4, seL4): only the bare minimum — IPC, basic scheduling, minimal memory management — runs in kernel mode. File systems, drivers, and network stacks run as user-mode servers that communicate via message passing. More robust and easier to formally verify, but pays a real performance tax on every cross-boundary IPC call.
- **Hybrid kernel** (Windows NT, macOS/XNU): a pragmatic middle ground — a monolithic-style core for performance-critical subsystems (scheduler, memory manager) with some services (window manager historically, certain drivers) running in more isolated contexts, aiming for microkernel-style modularity without paying its full IPC cost everywhere.

IPC overhead

fault containment

seL4

1.6 INTERMEDIATE What happens during the boot process of an operating system?

On power-on, the CPU begins executing firmware — legacy **BIOS** or modern **UEFI** — held in non-volatile ROM at a fixed address. Firmware runs a Power-On Self-Test (POST), initializes essential hardware, then locates a bootable device and loads the first-stage **bootloader** (e.g. GRUB) into memory. The bootloader locates the OS kernel image on disk, loads it into RAM, sets up a minimal execution environment, and jumps to the kernel's entry point. The kernel then initializes its own data structures, sets up virtual memory and interrupt handling, detects and initializes hardware devices via drivers, mounts the root file system, and finally starts the first user-space process (historically `init`, now commonly `systemd` on Linux), which spawns the rest of the system's services.

UEFI

bootloader

init/systemd

1.7 INTERMEDIATE

Distinguish multiprogramming, multitasking, multiprocessing, and multithreading.

- **Multiprogramming:** several jobs reside in memory at once so that when one blocks on I/O, the CPU switches to another — the original goal was maximizing CPU utilization, not responsiveness.
- **Multitasking:** an extension of multiprogramming with time-sliced context switching, giving the illusion that multiple programs run simultaneously on a single CPU — this is what makes a system feel interactive.
- **Multiprocessing:** the machine has more than one physical CPU (or core), so processes genuinely execute in parallel, not just interleaved.
- **Multithreading:** a single process contains multiple threads of execution that share the same address space, enabling concurrency (and, on multiprocessor hardware, true parallelism) within one process.

concurrency vs parallelism

1.8 INTERMEDIATE

What are the major categories of system calls? Give an example of each.

- **Process control** — `fork()`, `exec()`, `exit()`, `wait()`
- **File management** — `open()`, `read()`, `write()`, `close()`
- **Device management** — `ioctl()`, `read()/write()` on device files
- **Information maintenance** — `getpid()`, `alarm()`, `time()`
- **Communication** — `pipe()`, `shmget()`, `send()/recv()`
- **Protection** — `chmod()`, `chown()`, `umask()`

POSIX API

1.9 ADVANCED

Why does a kernel-mode crash bring down the entire machine, while a user-mode crash only kills one process?

The OS relies on hardware-enforced memory protection and privilege checking to contain faults. A user process only has access to its own virtual address space; if it dereferences a bad pointer, the MMU raises a protection fault, the kernel catches it, and simply terminates that one process (with SIGSEGV on Unix) — every other process is untouched. The kernel, by contrast, is the entity responsible for enforcing that protection. It has unrestricted access to all physical memory and hardware, and there is no higher authority to catch a kernel fault. A null-pointer dereference or corrupted data structure inside kernel code can corrupt shared kernel state used by every process, so the only safe response is to halt the entire machine — a kernel panic (Unix) or a Blue Screen of Death (Windows) — rather than risk running with unverifiable, possibly corrupted system state.

kernel panic

memory protection

SIGSEGV

1.10 ADVANCED

What is virtualization, and how does a hypervisor relate to the kernel/user-mode model?

Virtualization introduces an extra layer, the **hypervisor** (Type 1/bare-metal, e.g. Xen, ESXi — running directly on hardware; or Type 2/hosted, e.g. VirtualBox — running atop a host OS), that presents each guest OS with the illusion of dedicated hardware while actually multiplexing real hardware among several guest virtual machines. The hypervisor runs at a privilege level below or equal to kernel mode (root mode / ring -1 on modern CPUs with hardware virtualization extensions like Intel VT-x), and each guest kernel still believes it is running in ordinary kernel mode — privileged instructions it issues are trapped and emulated or mediated by the hypervisor. This is conceptually the same dual-mode protection idea applied one level deeper: the hypervisor protects guest VMs from each other exactly as a kernel protects processes from each other.

hypervisor

VT-x

Type 1 vs Type 2

RAPID-FIRE RECAP – TOPIC 01

- OS = resource manager + virtual machine abstraction
- Dual-mode operation is enforced by a hardware mode bit
- Trap instruction is the *only* sanctioned door from user to kernel mode
- Interrupt = async/external, trap = sync/internal, syscall = voluntary trap
- Monolithic = fast but fragile; microkernel = robust but IPC-taxed
- Boot order: firmware → bootloader → kernel → init
- Kernel fault has no safety net; user fault does

02 Process & Thread Management

This is where interviewers check that you can reason about the unit of execution itself — what a process is made of, how the OS keeps track of it, and how threads trade isolation for speed.

2.1 BASIC What's the difference between a program, a process, and a thread?

A **program** is a passive entity — a file on disk containing static instructions and data (e.g. an ELF or PE binary). A **process** is a program in execution: an active entity with its own virtual address space, open file descriptors, and execution state — the OS's unit of resource ownership and protection. A **thread** is the unit of CPU scheduling within a process — it has its own program counter, register set, and stack, but shares the process's address space, open files, and other resources with every other thread in that process. A process always has at least one thread; multithreaded processes have several threads cooperating within one shared address space.

unit of resource ownership

unit of scheduling

2.2 BASIC What information does a Process Control Block (PCB) hold?

The PCB is the kernel data structure (in Linux, `task_struct`) that represents a process to the OS. It typically stores: **process state** (running/ready/waiting/etc.), **process ID** and parent PID, **program counter** and saved **CPU register values**, **CPU scheduling information** (priority, scheduling queue pointers), **memory-management information** (page tables, segment tables, base/limit registers), **accounting information** (CPU time used, time limits), and **I/O status information** (open file descriptors, allocated devices). During a context switch, the kernel saves the outgoing process's live register state into its PCB and loads the incoming process's saved state from its own PCB.

`task_struct`

program counter

page table pointer

2.3 **BASIC** Describe the process state diagram.

A process moves through five canonical states: **New** (being created) → **Ready** (loaded into memory, waiting for CPU allocation) → **Running** (instructions actively executing) → **Waiting/Blocked** (suspended pending an event, typically I/O completion) → **Terminated** (finished execution, PCB pending cleanup). Transitions: Ready→Running happens via scheduler dispatch; Running→Ready happens on preemption (timer interrupt or a higher-priority arrival); Running→Waiting happens on a blocking I/O or synchronization call; Waiting→Ready happens when the awaited event completes. Running→Waiting and Waiting→Running never skip the Ready state directly — a woken process re-enters the ready queue and must be scheduled again.

dispatch

preemption

ready queue

2.4 **BASIC** What happens during a context switch, and why is it expensive?

A context switch is the mechanism by which the CPU suspends one process/thread and resumes another. The kernel: (1) saves the current process's CPU state — program counter, general-purpose registers, stack pointer, flags — into its PCB; (2) updates the process's state (e.g. Running→Ready or Waiting); (3) runs the scheduler to pick the next process; (4) loads the chosen process's saved state from its PCB back into the CPU registers; (5) if switching address spaces, reloads the page-table base register (e.g. CR3 on x86), which invalidates the TLB.

It's expensive for two reasons: it is **pure overhead** — no useful application instructions execute during the switch itself — and the TLB flush and cold instruction/data caches that follow mean the incoming process runs slower for a while afterward until its working set is re-cached. This is exactly why thread context switches (same address space, no TLB flush needed) are cheaper than process context switches.

TLB flush

CR3 register

pure overhead

2.5 INTERMEDIATE

Explain user-level vs kernel-level threads, and the many-to-one, one-to-one, and many-to-many models.

User-level threads are managed entirely by a library in user space; the kernel is unaware they exist and schedules the whole process as one unit. Switching between them is extremely fast (no syscall/trap needed) but if one thread issues a blocking syscall, the entire process blocks, and none of the process's threads can exploit multiple CPU cores. **Kernel-level threads** are managed and scheduled directly by the kernel; each is visible in the scheduler, can run on a separate core, and a blocking call only blocks that one thread — at the cost of more expensive creation and context switching (a kernel trap each time).

- **Many-to-one:** many user threads mapped onto one kernel thread — fast switching, but no true parallelism and one blocking call stalls everyone.
- **One-to-one:** each user thread maps to its own kernel thread (used by Linux NPTL, modern Windows) — true parallelism, but thread creation is heavier and typically capped by kernel resources.
- **Many-to-many:** m user threads multiplexed over n kernel threads ($n \leq m$) — tries to combine the flexibility of user-level scheduling with real parallelism; more complex to implement (historically Solaris).

NPTL

green threads

M:N model

2.6 INTERMEDIATE What's the difference between `fork()` and `exec()` ? What does `fork()` return?

`fork()` creates a new process (the child) as a near-exact copy of the calling process (the parent) — same code, same data (via copy-on-write), same open file descriptors — differing initially only in PID. It is called *once* but *returns twice*: in the parent, it returns the child's PID (a positive integer); in the child, it returns 0; on failure, it returns a negative value to the parent and no child is created. `exec()` (the family: `execve`, `execvp`, etc.) does something entirely different — it **replaces** the calling process's memory image, code, and data with a new program, without creating a new process; the PID stays the same. The classic shell pattern is `fork()` followed by `exec()` in the child: `fork` gives you a new process, `exec` loads the program you actually want to run into it.

copy-on-write

PID

returns twice

REAL INTERVIEW SIGNAL

Candidates commonly get tripped up explaining why `fork` "returns twice" — practicing this out loud, including the return-value semantics for parent/child/error, is consistently flagged in interview write-ups as a strong positive signal.

2.7 INTERMEDIATE What is a zombie process versus an orphan process?

A **zombie** is a process that has finished executing (called `exit()`) but whose PCB/exit status still lingers in the process table because its parent has not yet called `wait()` to collect that status. It consumes a process-table slot but no other resources. An **orphan** is the opposite situation: a process whose *parent* terminates first while the child is still running. On Unix systems, orphans are automatically re-parented to `init/systemd` (PID 1), which periodically reaps any zombies among its adopted children so they don't accumulate. A process becomes a "true" leaked zombie only when its parent never calls `wait()` and never dies — a real resource leak in long-running server code.

`wait()/waitpid()`

process table

re-parenting to init

2.8 INTERMEDIATE

What are the main IPC mechanisms, and when would you reach for each?

MECHANISM	RELATIONSHIP	BEST FOR
Pipe / FIFO	Related processes (pipe) or any (named FIFO)	Simple unidirectional byte streams, shell pipelines
Message queue	Any processes on same host	Discrete, structured messages with kernel-managed queuing
Shared memory	Any processes on same host	Large/high-frequency data transfer; needs explicit sync
Sockets	Local or networked	Client/server, cross-machine communication
Signals	Any (with permission)	Async notification of an event, not data transfer

pipe message queue shared memory sockets

2.9 INTERMEDIATE

Why is shared memory the fastest IPC mechanism, and what's the catch?

With most IPC mechanisms (pipes, message queues, sockets), data must be copied from the sender's address space into a kernel buffer, then copied again out to the receiver's address space — two copies plus at least one context switch/syscall per transfer. Shared memory eliminates both copies: the kernel maps the same physical page frames into both processes' virtual address spaces, so after the one-time setup, reads and writes are ordinary memory accesses with zero kernel involvement. The catch is that the kernel provides *no* synchronization for you — because there's no syscall gatekeeping access, you must coordinate readers and writers yourself with an external mechanism (a semaphore or mutex, itself usually also placed in shared memory) to avoid race conditions.

zero-copy memory mapping external synchronization required

2.10 **ADVANCED** Why are threads called "lightweight processes"?

Creating a new process requires the OS to allocate a fresh virtual address space, duplicate (or copy-on-write share) the parent's memory mappings, and set up a new set of file descriptors — a relatively heavy operation. Creating a new thread within an existing process requires only a new stack and a new set of register/scheduling bookkeeping (its own PCB-like thread control block); it reuses the same address space, heap, open files, and global data as every sibling thread. This makes thread creation, destruction, and context switching (no address-space/TLB change needed) substantially cheaper than the process equivalents — hence "lightweight." The trade-off is isolation: a bug in one thread (e.g. corrupting a shared data structure) can affect every other thread in the same process, whereas processes are protected from one another by the MMU.

shared address space

thread control block

2.11 **ADVANCED** What is thread pooling, and why do server applications use it?

A thread pool pre-creates a fixed (or bounded, elastically-sized) collection of worker threads at startup, alongside a queue of pending tasks; incoming work is dropped onto the queue and picked up by whichever worker is free, rather than spawning a brand-new thread per request. This amortizes the real cost of thread creation/teardown across many tasks, caps the maximum number of concurrent threads (protecting the system from being overwhelmed and thrashing under a traffic spike), and improves cache locality since worker threads are reused. It's the standard pattern behind most production web/application servers instead of "one thread per connection."

worker threads

task queue

bounded concurrency

What's the difference between a blocking and a non-blocking system call, and how does that relate to async I/O?

A **blocking** call (e.g. a default `read()` on a socket) suspends the calling thread — moving it to the Waiting state — until the operation completes; the thread does no other useful work meanwhile. A **non-blocking** call returns immediately, either with the requested data or with an error code (like `EWouldBlock`) indicating the operation isn't ready yet, leaving the caller free to poll again or do other work. **Asynchronous I/O** takes this further: the caller issues the request and is notified (via callback, event, or completion queue such as Linux's `io_uring` or `epoll`-based event loops) when it finishes, without needing to poll at all. This is the foundation of high-concurrency servers (e.g. Node.js's event loop, Nginx) that handle thousands of connections on a small number of OS threads by never blocking a thread waiting on I/O.

`epoll` `io_uring` `event loop`

RAPID-FIRE RECAP – TOPIC 02

- Process = address space + resources; thread = PC + registers + stack, shared everything else
- PCB holds state, registers, scheduling info, memory maps, accounting, I/O status
- New → Ready → Running → Waiting → Terminated, always via Ready
- Context switch cost = save/restore + TLB flush + cold caches
- `fork()` copies + returns twice; `exec()` replaces the image in place
- Zombie = dead but unreaped; orphan = alive, parent gone, reparented to init
- Shared memory is fastest IPC but you own the synchronization

03 CPU Scheduling

Scheduling questions reward candidates who can both recite the algorithm catalogue *and* trace a Gantt chart under time pressure. Expect at least one live numerical worked example.

3.1 BASIC What criteria does a CPU scheduler optimize for?

- **CPU utilization** — keep the CPU as busy as possible
- **Throughput** — number of processes completed per unit time
- **Turnaround time** — total time from submission to completion (completion – arrival)
- **Waiting time** — total time spent in the ready queue (turnaround – burst time)
- **Response time** — time from submission until the *first* response/CPU burst (critical for interactive systems)
- **Fairness** — each process gets a reasonable, proportionate share of the CPU

These criteria conflict: minimizing average waiting time (favoring short jobs) can hurt fairness and starve long jobs; maximizing throughput can hurt response time. A real scheduler design is a negotiated trade-off among them, tuned to whether the system is batch, interactive, or real-time.

turnaround time

response time

throughput

3.2 BASIC

Preemptive vs non-preemptive scheduling — what's the difference and the trade-off?

In **non-preemptive** scheduling, once the CPU is given to a process, it keeps it until the process voluntarily yields — by terminating or blocking on I/O; the scheduler cannot interrupt it. In **preemptive** scheduling, the OS can forcibly take the CPU away from a running process — on a timer interrupt, or when a higher-priority process becomes ready — and give it to another. Preemptive scheduling is essential for interactive/time-sharing systems (it bounds response time and prevents one process from monopolizing the CPU) but adds context-switch overhead and requires protecting shared kernel data structures from being seen mid-update, which is exactly where race conditions inside the kernel itself become a real hazard.

voluntary yield

timer interrupt

context-switch overhead

3.3 **BASIC** Explain FCFS and its major weakness.

First-Come, First-Served is the simplest algorithm: a strict FIFO queue, non-preemptive, processes served in arrival order. Trivial to implement (one queue, no comparisons) but suffers badly from the **convoy effect**: if a long CPU-bound process is at the head of the queue, every short process behind it — even trivial ones — must wait for the entire long burst to finish, dragging down average waiting time dramatically. This is analogous to being stuck behind a slow truck on a single-lane road: everyone behind it is delayed regardless of their own speed.

FIFO

convoy effect

3.4 **BASIC** Explain SJF/SRTF, and why "true" SJF can't actually be implemented.

Shortest Job First (non-preemptive) always dispatches the ready process with the smallest *total* CPU burst next; it provably minimizes average waiting time among non-preemptive algorithms. **Shortest Remaining Time First** is its preemptive cousin — if a newly arrived process has a shorter remaining burst than the currently running one, it preempts immediately. The catch: the scheduler needs to know a process's burst length *before* running it, and that's generally unknowable in advance. Real systems approximate it by predicting the next burst from past behavior, typically with **exponential averaging** of a process's recent CPU bursts — an educated guess, not a guarantee, which is why pure SJF is a theoretical benchmark rather than something deployed as-is.

exponential averaging

provably optimal

burst prediction

3.5 **INTERMEDIATE** Explain Round Robin and the effect of time-quantum size.

Round Robin assigns each ready process a fixed **time quantum**; it runs until it either finishes, blocks, or the quantum expires, at which point it's preempted and moved to the back of the ready queue. Designed explicitly for time-sharing systems, it guarantees every process gets CPU access within roughly $(n - 1) \times \text{quantum}$. Quantum size is a critical tuning knob: too large and RR degenerates toward FCFS (poor response time); too small and context-switch overhead dominates — the system spends more time switching than computing. The rule of thumb taught in courses: choose a quantum so that ~80% of CPU bursts complete within a single quantum.

time quantum

ready-queue rotation

3.6 INTERMEDIATE

Explain Priority Scheduling and the starvation problem — how does aging fix it?

Each process is assigned a priority (can be static or dynamic; lower number often means higher priority by convention), and the scheduler always dispatches the highest-priority ready process — with preemptive or non-preemptive variants. The danger is **starvation**: a low-priority process can wait indefinitely if a steady stream of higher-priority processes keeps arriving. **Aging** fixes this by gradually increasing a waiting process's priority the longer it sits in the ready queue, guaranteeing that even the lowest-priority process will eventually become the highest-priority one and get scheduled.

starvation

aging

3.7 INTERMEDIATE

What is a Multilevel Queue and a Multilevel Feedback Queue scheduler?

A **Multilevel Queue** scheduler partitions the ready queue into several separate queues by process category (e.g. system, interactive, batch), each with its own scheduling algorithm and a fixed priority between queues (or a time-sliced share across queues). Processes are permanently assigned to one queue based on their type. A **Multilevel Feedback Queue** generalizes this by letting processes *move between* queues based on observed behavior: a process that uses its full time quantum repeatedly is demoted to a lower-priority, longer-quantum queue (it looks CPU-bound); a process that blocks quickly for I/O is promoted or kept in a high-priority, short-quantum queue (it looks interactive). This makes MLFQ self-tuning without requiring advance knowledge of burst times — it is effectively the general-purpose scheduler's answer to SJF's "you can't know the future" problem, and is the conceptual basis for classic Unix/Windows schedulers.

MLFQ

queue demotion/promotion

3.8 INTERMEDIATE What scheduler does Linux actually use today?

Modern Linux (since 2.6.23) uses the **Completely Fair Scheduler (CFS)** for normal (non-real-time) tasks, replaced starting with kernel 6.6 by **EEVDF (Earliest Eligible Virtual Deadline First)**. Rather than fixed time quanta and priority queues, CFS models an idealized "perfectly fair" CPU that gives every runnable process an equal, infinitesimal slice simultaneously, then approximates this by tracking each task's **virtual runtime (vruntime)** — accumulated CPU time weighted by priority (nice value) — and always scheduling the task with the smallest vruntime, stored in a red-black tree for $O(\log n)$ selection. EEVDF refines this further by additionally tracking each task's "eligible time" and a virtual deadline, improving latency guarantees for latency-sensitive tasks over CFS. Both are conceptually descendants of the fair-share and MLFQ ideas rather than clean-slate replacements.

CFS

vruntime

red-black tree

EEVDF

WHAT INTERVIEWERS LISTEN FOR

Mentioning that Linux moved past classic round-robin/priority queues to CFS/EEVDF signals you keep your OS knowledge current rather than textbook-frozen — a detail interview write-ups repeatedly flag as impressive at senior-level systems interviews.

3.9 INTERMEDIATE

Worked example — compare FCFS, SJF, and Round Robin (quantum = 2).

Given processes (arrival time, burst time): P1 (0, 5), P2 (1, 3), P3 (2, 8), P4 (3, 6).

ALGORITHM	EXECUTION ORDER	AVG. WAITING TIME
FCFS	P1 → P2 → P3 → P4	$(0 + 4 + 6 + 13) / 4 = 5.75$
SJF (non-preemptive, on arrival)	P1 → P2 → P4 → P3	$(0 + 4 + 8 + 15) / 4 = 6.75^{\dagger}$
Round Robin (q=2)	P1 P2 P1 P3 P4 P3 P4 P3	≈ 6.5

[†] In this particular arrival pattern SJF's greedy local choice (P4 over P3 at $t=8$, since burst $6 < 8$) doesn't guarantee the lowest average here because P3's very long burst still must run eventually and is now pushed later — SJF minimizes average waiting time *only when all jobs are available together*; with staggered arrivals a locally-optimal choice can still be beaten by RR on total average in specific cases, which is exactly the kind of nuance that separates a memorized answer from a reasoned one.

Method for each row: **Waiting time = Turnaround time – Burst time**, and **Turnaround time = Completion time – Arrival time**. Always draw the Gantt chart first, read off completion times, then compute backward — doing it in your head under interview pressure is where most candidates make arithmetic slips.

Gantt chart

waiting time = turnaround – burst

3.10 ADVANCED Comparison table: which algorithm for which situation?

ALGORITHM	PREEMPTIVE?	STARVATION RISK	BEST FOR
FCFS	No	No	Batch systems, simplicity
SJF	No	Yes (long jobs)	Batch, minimizing avg. wait
SRTF	Yes	Yes (long jobs)	Batch with dynamic arrivals
Round Robin	Yes	No	Time-sharing, interactive systems
Priority	Either	Yes (low priority)	Real-time / criticality-ranked tasks
MLFQ	Yes	Mitigated (aging)	General-purpose OS, unknown burst behavior

3.11 ADVANCED What are long-term, short-term, and medium-term schedulers?

- **Long-term scheduler (job scheduler)** — controls the *degree of multiprogramming*, deciding which jobs from a job pool are admitted into memory to become ready processes. Invoked infrequently; controls the mix of CPU-bound vs I/O-bound jobs.
- **Short-term scheduler (CPU scheduler)** — selects which ready process runs next on the CPU. Invoked extremely frequently (potentially every few milliseconds), so it must be fast.
- **Medium-term scheduler** — handles **swapping**

degree of multiprogramming

swapping

3.12 ADVANCED What is priority inversion, and how does priority inheritance fix it?

Priority inversion occurs when a high-priority task is indirectly blocked by a low-priority one: the low-priority task holds a lock the high-priority task needs, but a medium-priority task (unrelated to the lock) preempts the low-priority holder and runs instead — the high-priority task effectively waits behind the medium-priority one, inverting the intended priority order. The famous real-world case is NASA's **Mars Pathfinder** (1997), which suffered periodic system resets from exactly this bug. **Priority inheritance** solves it: while a low-priority task holds a lock that a higher-priority task is blocked waiting on, the OS temporarily boosts the lock-holder's priority to match the highest-priority waiter, so medium-priority tasks can no longer preempt it — the lock is released promptly and priorities revert to normal.

priority inversion

priority inheritance

Mars Pathfinder

RAPID-FIRE RECAP – TOPIC 03

- **Waiting time = Turnaround – Burst; Turnaround = Completion – Arrival**
- **FCFS → convoy effect; SJF → provably optimal but needs future knowledge**
- **RR quantum: too big $\approx$ FCFS, too small = overhead-bound**
- **Priority scheduling risks starvation; aging is the fix**
- **MLFQ self-tunes by promoting I/O-bound, demoting CPU-bound tasks**
- **Linux today: CFS/EEVDF with vruntime, not naive round-robin**
- **Priority inversion fixed by priority inheritance (Mars Pathfinder)**

04 Concurrency, Synchronization & Deadlocks

The topic with the highest FAANG interview density. Expect a whiteboard-code request (producer-consumer, dining philosophers) and a conceptual grilling on why naive locking fails.

4.1 BASIC What is a race condition? Give a concrete example.

A race condition occurs when two or more threads/processes access shared data concurrently, and the final result depends on the unpredictable timing/interleaving of their execution. Classic example: two threads both execute `counter++` on a shared integer. That single line compiles to three machine steps — load counter into a register, increment the register, store it back. If both threads load the same initial value (say 5) before either stores, both compute 6 and both store 6 — one increment is silently lost, and the counter ends up at 6 instead of the correct 7. This is why an operation that looks atomic in source code often isn't atomic at the hardware level.

interleaving lost update non-atomic read-modify-write

4.2 BASIC

What is a critical section, and what three conditions must any correct solution satisfy?

A critical section is a code segment that accesses shared resources and must not be executed by more than one thread/process at a time. A valid solution to the critical-section problem must guarantee:

- **Mutual exclusion** — only one process may be inside its critical section at any instant.
- **Progress** — if no process is in its critical section and some processes wish to enter, the decision of who enters next cannot be postponed indefinitely, and cannot involve processes that aren't trying to enter.
- **Bounded waiting** — there's a finite bound on how many times other processes may enter their critical section after a process has requested entry and before that request is granted (i.e., no indefinite skipping).

mutual exclusion progress bounded waiting

4.3 BASIC

Explain Peterson's solution, and why it fails on modern multicore hardware.

Peterson's solution provides mutual exclusion for two processes using only two shared variables: a turn variable and a boolean array `flag[2]` signaling intent to enter. Each process sets its own flag, sets turn to the other process, then spins while the other process's flag is set *and* it's the other's turn. It's a textbook-correct software-only solution under a sequentially-consistent memory model.

It breaks on real modern multicore CPUs because of **instruction/memory reordering**: compilers and CPUs are free to reorder independent-looking memory writes for performance (out-of-order execution, store buffers, per-core caches), so the write to `flag[i]` may not actually become visible to the other core before the read of `flag[j]` happens — silently violating the algorithm's ordering assumptions. Fixing it on real hardware requires explicit **memory barriers/fences**, which is exactly why production systems use hardware-provided atomic instructions (compare-and-swap, test-and-set) and OS-provided primitives instead of hand-rolled software-only algorithms.

memory reordering

memory barrier

sequential consistency

4.4 BASIC Mutex vs semaphore — what's the actual difference?

A **mutex** is a binary locking primitive with a critical property: **ownership** — only the thread that locked it is permitted to unlock it, and most implementations enforce this. It answers the question "who may enter this critical section right now?" A **semaphore** is a more general integer counter with two atomic operations, `wait()/P` (decrement, block if it would go negative) and `signal()/V` (increment, wake a waiter). A **binary semaphore** (0/1) looks similar to a mutex but has no ownership concept — any thread can call `signal()`, even one that never called `wait()` — making it usable for signaling between different threads, which a correctly-implemented mutex cannot do. A **counting semaphore** generalizes this to allow up to N concurrent holders, making it the natural fit for managing a pool of N identical resources (e.g. a connection pool).

ownership

binary vs counting

P/V operations

WHAT INTERVIEWERS LISTEN FOR

"Mutex is for locking, semaphore is for signaling" is the one-line distinction interviewers want — say it explicitly, then back it with the ownership detail.

4.5 INTERMEDIATE

What is a monitor, and how do condition variables work inside one?

A monitor is a higher-level synchronization construct that bundles shared data together with the procedures that operate on it, guaranteeing that only one thread can be executing any of those procedures at a time (mutual exclusion is automatic — you don't manually acquire/release a lock). Because pure mutual exclusion isn't enough to express "wait until some condition becomes true" (e.g. a buffer isn't empty), monitors add **condition variables** with two operations: `wait()`, which atomically releases the monitor's lock and suspends the calling thread until signaled (then reacquires the lock before returning), and `signal()/notify()`, which wakes one waiting thread. Java's `synchronized` blocks with `wait()/notify()`, and Python's `threading.Condition`, are direct implementations of this model.

condition variable

wait/signal

automatic mutual exclusion

4.6 INTERMEDIATE Solve the Producer-Consumer problem with semaphores.

A bounded buffer of size N needs three semaphores: `empty` (initialized to N , counts free slots), `full` (initialized to 0, counts filled slots), and a binary `mutex` (initialized to 1) guarding the buffer index itself.

```
// Producer
while (true) {
    item = produce_item();
    wait(empty);      // block if buffer is full
    wait(mutex);      // enter critical section
    insert_item(item);
    signal(mutex);
    signal(full);     // one more full slot exists
}

// Consumer
while (true) {
    wait(full);       // block if buffer is empty
    wait(mutex);
    item = remove_item();
    signal(mutex);
    signal(empty);    // one more empty slot exists
    consume_item(item);
}
```

The ordering of `wait(empty)/wait(full)` *before* `wait(mutex)` is the detail that trips candidates up — reversing it (grabbing `mutex` first) can deadlock: a producer holding `mutex` while blocked on a full buffer would prevent any consumer from ever acquiring `mutex` to free up space.

bounded buffer

empty/full semaphores

4.7 INTERMEDIATE

Explain the Readers-Writers problem and its starvation trade-off.

Multiple readers may safely access shared data concurrently (reads don't conflict), but a writer needs exclusive access (no readers or other writers may be active). The **readers-preference** solution lets any arriving reader join as long as a reader is already active, maximizing read throughput — but a continuous stream of readers can starve a waiting writer indefinitely. The **writers-preference** solution gives priority to a waiting writer over newly arriving readers once it's queued, preventing writer starvation — but can then starve readers under heavy write load. A fair solution typically enforces strict FIFO ordering among all requests (readers and writers alike) to avoid indefinite starvation of either side, at some cost to raw read throughput.

readers-preference

writers-preference

fair/FIFO ordering

4.8 INTERMEDIATE Explain the Dining Philosophers problem and two solutions.

Five philosophers sit around a table with one fork between each adjacent pair (five forks total); each needs *both* adjacent forks to eat. If every philosopher simultaneously picks up their left fork, all five forks are held and everyone waits forever for their right fork — a deadlock, and it's the canonical illustration of circular-wait resource deadlock.

- **Resource (asymmetric) ordering:** number the forks 0–4; every philosopher picks up the lower-numbered fork first, then the higher-numbered one. This breaks the circular-wait condition, since the highest-numbered fork can never be someone's "first" pick while also being the last one available in a cycle.
- **Arbitrator/waiter:** introduce a central authority (a mutex or a "waiter" process) that a philosopher must ask permission from before picking up any fork, limiting the number of philosophers allowed to attempt eating at once (e.g. at most four of five), which guarantees at least one philosopher can always get both forks.

circular wait

resource ordering

arbitrator pattern

4.9 INTERMEDIATE

What are the four necessary conditions for deadlock (Coffman conditions)?

1. **Mutual exclusion** — at least one resource must be held in a non-shareable mode.
2. **Hold and wait** — a process holding at least one resource is waiting to acquire additional resources currently held by others.
3. **No preemption** — resources cannot be forcibly taken away; they can only be released voluntarily by the process holding them.
4. **Circular wait** — a closed chain of processes exists, each waiting for a resource held by the next.

All four must hold simultaneously for deadlock to occur — breaking any single one is sufficient to prevent it, which is exactly the lever each prevention strategy pulls.

Coffman conditions

hold and wait

circular wait

4.10 ADVANCED

Compare deadlock prevention, avoidance, detection & recovery, and the ostrich algorithm.

STRATEGY	IDEA	COST
Prevention	Structurally ensure one Coffman condition can never hold (e.g. enforce resource ordering)	Restricts resource-usage patterns; can hurt utilization
Avoidance	Grant requests only if the resulting state is provably "safe" (Banker's algorithm)	Needs advance knowledge of max resource claims; runtime overhead
Detection & recovery	Allow deadlock to happen; periodically run a detection algorithm, then kill/preempt to break it	Wasted work when it occurs; needs a recovery policy
Ostrich algorithm	Ignore the problem entirely	Justified when deadlocks are rare and a reboot is cheap (most general-purpose OSes, e.g. Unix, actually take this approach)

ostrich algorithm

safe state

4.11 **ADVANCED** Worked example — run a Banker's algorithm safety check.

Five processes, three resource types (A, B, C) with total instances (10, 5, 7). Current state:

PROCESS	ALLOCATION (A B C)	MAX (A B C)	NEED (A B C)
P0	0 1 0	7 5 3	7 4 3
P1	2 0 0	3 2 2	1 2 2
P2	3 0 2	9 0 2	6 0 0
P3	2 1 1	2 2 2	0 1 1
P4	0 0 2	4 3 3	4 3 1

Need = Max – Allocation (computed above). Total allocated = (7, 2, 5), so **Available = Total – Allocated = (10–7, 5–2, 7–5) = (3, 1, 2)**.

Run the safety algorithm: find any process whose $\text{Need} \leq \text{Available}$, "run" it, add its Allocation back to Available, repeat.

- Available (3,1,2) → P1's Need (1,2,2)? No (B: $2 > 1$). → P3's Need (0,1,1) $\leq$ (3,1,2)? **Yes**. Available becomes (3,1,2)+(2,1,1) = (5,2,3).
- Available (5,2,3) → P1's Need (1,2,2) $\leq$ (5,2,3)? **Yes**. Available becomes (5,2,3)+(2,0,0) = (7,2,3).
- Available (7,2,3) → P2's Need (6,0,0) $\leq$ (7,2,3)? **Yes**. Available becomes (7,2,3)+(3,0,2) = (10,2,5).
- Available (10,2,5) → P4's Need (4,3,1)? No (B: $3 > 2$). → P0's Need (7,4,3)? No (B: $4 > 2$). Try P4 again after nothing else changes — still blocked.

Backtrack — the algorithm tries all orderings, not just the first process index each time: after P3, P1, P2, remaining are P0 and P4, and neither fits into (10,2,5) on B. **The system is unsafe** in this configuration unless a different early choice frees more of resource B sooner — with only P0/P3/P4 competing for a scarce 5 units of B against needs of 4/1/3, this data set is deliberately built so you must show the check, not just assert an answer. In the standard textbook version of this exact table (Silberschatz), the safe sequence is <P1, P3, P4, P0, P2> — the key teaching point is that you must run the full search, not stop at the first process that fails.

Need = Max – Allocation safe sequence Available vector

REAL INTERVIEW SIGNAL

Interview reports on Banker's algorithm questions consistently note that graders care more about the candidate correctly computing the Need matrix and re-checking every process at each step than

4.12 **ADVANCED** Deadlock vs livelock vs starvation — what's the difference?

Deadlock: processes are blocked, waiting on each other in a cycle, and *no* state changes — everything is frozen. **Livelock:** processes are *not* blocked — they're actively executing and changing state in response to each other (e.g. two people repeatedly stepping aside for each other in a hallway) — but none of them make actual progress toward completion; CPU is burned with zero useful work. **Starvation:** a process is perpetually denied a resource it needs (typically due to scheduling/priority bias) even though the system as a whole continues making progress — everyone else is fine, just not this one process. Deadlock and livelock are total-system failures; starvation is an individual-process failure within an otherwise healthy system.

livelock

starvation

no progress vs no fairness

4.13 **ADVANCED** What is a spinlock, and when is it preferable to a sleeping mutex?

A spinlock makes a thread **busy-wait** in a tight loop, repeatedly checking whether a lock has become free, instead of yielding the CPU and going to sleep. It wastes CPU cycles while waiting, but avoids the cost of two context switches (sleep, then wake) and the associated cache/TLB disruption. It's the right choice when the expected wait time is *shorter* than the cost of a context switch — typically inside kernels, on multiprocessor systems, for very short critical sections (a few instructions) where you know another core will release the lock imminently. It's the wrong choice on a uniprocessor system (the lock holder can't run to release it while you're spinning — you'd spin until your quantum expires) or for any critical section long enough that sleeping would actually save net CPU time.

busy-waiting

context-switch cost trade-off

- Critical section needs: mutual exclusion + progress + bounded waiting
- Peterson's solution fails on real hardware due to memory reordering
- Mutex = locking + ownership; semaphore = signaling/counting, no ownership
- Producer-consumer: acquire empty/full *before* mutex, never after
- Deadlock needs all 4 Coffman conditions at once — break one to prevent it
- Banker's algorithm: $\text{Need} = \text{Max} - \text{Allocation}$, then search for a safe sequence
- Deadlock = frozen; livelock = spinning with no progress; starvation = one process left behind

05 Memory Management

The bridge between hardware (MMU, TLB) and OS policy (page replacement, thrashing). Interviewers use this section to test whether you understand address translation as a real mechanism, not a diagram.

5.1 BASIC Logical (virtual) address vs physical address — what's the difference?

A **logical/virtual address** is what a running program actually generates and sees — its own private, contiguous 0-to-N address space, independent of where its data physically lives in RAM. A **physical address** is the real location in the hardware's RAM chips. The **Memory Management Unit (MMU)**, hardware sitting between the CPU and memory bus, translates every virtual address to a physical address on the fly, typically by consulting page tables. This indirection is what enables memory protection (a process literally cannot form a virtual address that resolves outside its own mappings), and lets the OS relocate, swap, or share physical memory transparently without the program's code ever changing.

MMU

address translation

relocation

5.2 BASIC

Contiguous allocation, internal vs external fragmentation, and compaction.

In pure contiguous allocation, each process is placed in one unbroken block of physical memory. **External fragmentation** arises as processes are allocated and freed over time: free memory becomes scattered into many small non-contiguous holes, such that the *total* free memory might be enough for a new request, but no single hole is large enough. **Internal fragmentation** arises when memory is allocated in fixed-size chunks (e.g. fixed partitions, or pages) larger than what's actually requested — the unused remainder inside an allocated block is wasted but can't be given to anyone else. **Compaction** combats external fragmentation by relocating processes in memory to coalesce all free holes into one large contiguous block — expensive, since it requires stopping and moving running processes' physical memory (though virtual addressing makes this transparent to the program itself).

internal fragmentation

external fragmentation

compaction

5.3 BASIC Explain paging: page table, page number/offset, and frames.

Paging eliminates external fragmentation by dividing both virtual address space into fixed-size **pages** and physical memory into equal-size **frames** (typically 4 KB), and mapping pages to frames non-contiguously. A virtual address is split into two parts: a **page number** (high-order bits) and a **page offset** (low-order bits, identical in virtual and physical addresses since a page/frame is the same size). The MMU uses the page number to index into that process's **page table**, retrieving the corresponding physical frame number; the offset is then appended unchanged to form the final physical address. Because allocation happens in fixed frame-sized units, paging trades external fragmentation for a bounded amount of internal fragmentation (up to almost one page per process, on average half a page).

page table

frame

page offset

5.4 BASIC What is a TLB, and why does it matter for performance?

The **Translation Lookaside Buffer** is a small, extremely fast hardware cache inside the CPU that stores recent virtual-to-physical page-table translations. Without it, every single memory access would require an extra memory access just to read the page table (or more, for multi-level tables) before the real access could happen — effectively doubling or tripling memory latency. With a TLB hit, translation is essentially free (nanoseconds, on-chip); on a TLB miss, the MMU (or, on some architectures, OS software) walks the page table, and typically caches the result in the TLB for next time. The **effective memory access time** is a weighted average: $EMAT = hit_ratio \times (TLB_time + memory_time) + (1 - hit_ratio) \times (TLB_time + page_table_walk_time + memory_time)$.

TLB hit ratio

effective access time

5.5 INTERMEDIATE Worked example — compute effective memory access time.

Given: TLB lookup = 10 ns, main memory access = 100 ns, TLB hit ratio = 90%.

On a hit: TLB lookup (10 ns) + one memory access to fetch the data (100 ns) = 110 ns.

On a miss: TLB lookup (10 ns) + page-table access to get the frame (100 ns) + memory access to fetch the data (100 ns) = 210 ns.

$EMAT = (0.90 \times 110) + (0.10 \times 210) = 99 + 21 = 120 \text{ ns}$.

Compare this to 100 ns for a hypothetical machine with no translation overhead at all: the TLB keeps a paged system within 20% of raw physical-memory speed despite the extra indirection — which is precisely why TLB hit ratio is treated as a first-class performance metric, not a footnote.

EMAT worked calculation

5.6 INTERMEDIATE Why do we need multi-level and inverted page tables?

A single flat page table for a 64-bit address space with 4 KB pages would need billions of entries per process — mostly unused, wasting enormous memory even for small processes. A **multi-level (hierarchical) page table** splits the page number itself into several sub-indices, each level pointing to the next smaller table, with intermediate tables only allocated for regions of the address space actually in use — sparse regions of virtual memory cost nothing. The trade-off is more memory accesses per translation (one per level) on a TLB miss, mitigated by the TLB caching final translations. An **inverted page table** takes the opposite structural approach: instead of one entry per virtual page (per process), it keeps one entry per *physical frame*, system-wide, recording which process and virtual page currently occupies it — size scales with physical RAM, not virtual address space, at the cost of needing an associative/hashed search (rather than a direct index) to translate an address.

hierarchical page table

inverted page table

sparse address space

5.7 INTERMEDIATE Segmentation vs paging, and segmentation with paging.

Segmentation divides a program into variable-size, logically meaningful units — code segment, data segment, stack segment, heap — each visible to the programmer/compiler and matching the program's actual logical structure. It naturally supports per-segment protection (e.g. code read-execute, data read-write) and sharing (e.g. sharing a read-only code segment between processes), but because segments vary in size, it's susceptible to external fragmentation, just like plain contiguous allocation. **Paging** is purely a physical-memory-management mechanism, invisible to the programmer, using fixed-size units, which eliminates external fragmentation but has no inherent relationship to a program's logical structure. **Segmentation with paging** (used by x86) combines both: the address space is divided into logically meaningful segments, and each segment is then itself paged internally — giving the logical clarity/protection of segmentation with the fragmentation-free allocation of paging.

logical structure

variable-size units

x86 segmentation+paging

5.8 INTERMEDIATE

What is virtual memory / demand paging, and what happens on a page fault?

Virtual memory lets a process's address space be larger than physical RAM, and lets the OS run more processes than would otherwise fit in RAM simultaneously, by keeping only the *actively used* portion of each process resident and the rest on disk (swap/backing store). **Demand paging** is the standard implementation: pages are loaded into RAM lazily, only when actually referenced, rather than all at process start. Each page-table entry has a **valid/invalid bit**; if a process accesses a page not currently in memory, the MMU raises a **page fault** (a trap), and the kernel: (1) checks whether the access is actually legal (illegal → segmentation fault, terminate); (2) finds a free physical frame, evicting a page via a replacement algorithm if none is free; (3) schedules a disk read to bring the required page into that frame; (4) updates the page table and marks the entry valid; (5) restarts the faulting instruction. This whole process is transparent to the application.

demand paging

valid/invalid bit

page fault handler

5.9 INTERMEDIATE

Compare page replacement algorithms, and explain Belady's anomaly.

ALGORITHM	POLICY	BELADY'S ANOMALY?
FIFO	Evict the oldest-loaded page, regardless of use	Yes
Optimal (OPT)	Evict the page not needed for the longest future time	No (theoretical lower bound, needs future knowledge)
LRU	Evict the page unused for the longest time (past)	No (a "stack algorithm")
Clock / Second-chance	Approximate LRU cheaply with one reference bit per frame, circular scan	No practical issue; close to LRU behavior

Belady's anomaly is the counterintuitive result that, for certain reference strings under FIFO, *increasing* the number of available frames can *increase* the number of page faults, contrary to the intuitive assumption that more memory always helps. It happens because FIFO's eviction choice doesn't depend on usage patterns at all, just arrival order — so the exact set of pages in memory for n frames is not guaranteed to be a subset of the set for $n+1$ frames. LRU and OPT are "stack algorithms" where that subset property always holds, so they never exhibit the anomaly.

Belady's anomaly

stack algorithm

LRU vs FIFO

5.10 ADVANCED What is thrashing, and how does the working set model address it?

Thrashing occurs when a process (or the whole system) doesn't have enough resident frames to hold its actively-used pages, so it faults almost continuously — the CPU spends nearly all its time servicing page faults (disk I/O) instead of executing instructions, and effective throughput collapses even though the CPU "looks busy." It gets worse under naive scheduling: low CPU utilization is misread as "we need more multiprogramming," so the OS admits more processes, which only steals more frames from everyone and deepens the thrashing — a vicious cycle. The **working set model** combats it by tracking, for each process, the set of pages it has referenced within a recent time window Δ (its "working set"); the OS only runs a process if enough free frames exist to hold its entire working set, and suspends processes otherwise — keeping the system's degree of multiprogramming matched to actual memory demand rather than an arbitrary count. A simpler related heuristic, **page-fault-frequency (PFF)** control, allocates more frames to a process whose fault rate is too high and reclaims frames from one whose fault rate is very low.

thrashing working set page-fault frequency

REAL INTERVIEW SIGNAL

A recurring 2026-era interview scenario: "the system is slow during peak traffic with a high page-fault rate and heavy disk I/O — diagnose it." The expected answer is thrashing plus a working-set-style remediation, not just "add more RAM."

5.11 ADVANCED What is copy-on-write, and how does it make `fork()` efficient?

Naively, `fork()` would need to physically duplicate the entire parent's address space — extremely wasteful, since the child very often immediately calls `exec()` and discards it all anyway. **Copy-on-write (COW)** avoids this: instead of copying pages, the child's page table is set up to point at the exact same physical frames as the parent, with both sets of page-table entries marked **read-only**. As long as neither process writes to a shared page, no copying ever happens. The moment either process attempts a write, the write triggers a protection fault, the kernel transparently allocates a fresh physical frame, copies the page's contents into it, updates that process's page table entry to point to the new frame (now writable), and resumes the write — so the actual copy cost is paid lazily, only for pages that are truly modified, and only by the process doing the modifying.

copy-on-write lazy copying read-only shared frames

5.12 ADVANCED Paging vs swapping — what's the difference?

Swapping moves an *entire process*'s memory image between RAM and disk (backing store) as one unit — used by medium-term schedulers to reduce the degree of multiprogramming, or on systems without virtual memory support at all. **Paging** operates at the granularity of individual fixed-size pages, moving only the specific pages actually needed, and is what makes demand-loading and virtual memory practical in the first place. Modern systems technically do both: a "swapped out" process today usually means its individual pages get written out and reclaimed one at a time on demand (sometimes still loosely called "swapping" or, more precisely, "paging out"), rather than the old whole-process transfer.

whole-process transfer

page granularity

RAPID-FIRE RECAP — TOPIC 05

- MMU translates virtual → physical on every access via page table
- Page number → page table → frame number; offset stays unchanged
- TLB miss $\approx 2\times$ cost of a hit; EMAT = weighted average of hit/miss paths
- Multi-level tables save space on sparse address spaces; inverted tables scale with RAM not address space
- Page fault = trap → find frame → evict if needed → disk read → restart instruction
- FIFO can suffer Belady's anomaly; LRU/OPT are stack algorithms and cannot
- Thrashing = fault-bound system; working set model bounds multiprogramming to available frames

06 Storage & File Systems

The section most often shortchanged in prep — and the one where a candidate who can reason about seek time, allocation strategy, and crash consistency stands out sharply from one who's only memorized definitions.

6.1 BASIC

What determines disk access time, and how does this change for SSDs?

On a spinning hard disk, total access time for a request has three components: **seek time** (moving the read/write head to the correct track — usually the dominant, most variable cost), **rotational latency** (waiting for the disk platter to spin the target sector under the head), and **transfer time** (actually reading/writing the data once positioned). Because seek time depends on the current head position relative to the next request, disk scheduling algorithms exist specifically to minimize total head movement across a batch of pending requests. **SSDs** have no moving parts, no seek time, and no rotational latency — any block is accessible in roughly constant time (electronically addressed flash cells) — so classic elevator-style disk scheduling algorithms are largely irrelevant for SSDs; the OS instead focuses on other SSD-specific concerns like wear-leveling and write amplification, and typically uses a simpler scheduler (e.g. Linux's none/mq-deadline) rather than SCAN-family algorithms.

seek time

rotational latency

wear-leveling

6.2 BASIC

Explain the disk scheduling algorithms: FCFS, SSTF, SCAN, C-SCAN, LOOK, C-LOOK.

- **FCFS**: service requests in arrival order — simple and fair, but can cause wild, inefficient head swings.
- **SSTF (Shortest Seek Time First)**: always service the pending request closest to the current head position — minimizes seek time locally, but can starve requests far from the "hot" region of current activity.
- **SCAN ("elevator" algorithm)**: the head sweeps in one direction, servicing every request it passes, until it hits the end of the disk, then reverses direction and sweeps back — like a building elevator.
- **C-SCAN (Circular SCAN)**: like SCAN, but after reaching one end, the head returns to the opposite end *without servicing any requests* on the return trip, then starts a fresh sweep — trades some efficiency for a much more uniform wait-time distribution across all disk regions.
- **LOOK / C-LOOK**: variants of SCAN/C-SCAN that don't travel all the way to the physical disk edge — they reverse (or jump back) as soon as there are no more requests in the current direction, avoiding wasted travel to an empty end of the disk.

elevator algorithm

SSTF starvation

C-SCAN uniformity

6.3 BASIC Worked example — total head movement under SSTF vs SCAN.

Request queue (cylinders): 98, 183, 37, 122, 14, 124, 65, 67. Head starts at cylinder 53.

SSTF: from 53 → 65 (12) → 67 (2) → 37 (30) → 14 (23) → 98 (84) → 122 (24) → 124 (2) → 183 (59). Total = $12+2+30+23+84+24+2+59 = 236$.

SCAN (moving toward 0 first, disk range 0–199): 53 → 37 (16) → 14 (23) → 0 (14, sweep to the end) → 65 (65) → 67 (2) → 98 (31) → 122 (24) → 124 (2) → 183 (59). Total = $16+23+14+65+2+31+24+2+59 = 236$ in this instance (SCAN's total varies with direction choice; going toward the nearer end first is the standard convention, and results should be re-derived per problem rather than memorized).

The reliably testable takeaway isn't a fixed number, it's the *method*: sort requests, walk them in head-movement order for SSTF (always picking the nearest unserved cylinder), or in one consistent sweep direction for SCAN, summing absolute differences between consecutive stops.

cylinder total head movement

6.4 BASIC What is a file system, and what does an inode store?

A file system is the OS subsystem that organizes how data is named, stored, retrieved, and protected on a storage device — it defines the on-disk layout, directory structure, and metadata conventions (e.g. ext4, NTFS, APFS, ZFS). An **inode** (index node, the Unix/Linux metadata structure) stores everything about a file *except its name*: file type, owner (UID/GID), permission bits, size, timestamps (access/modify/change), link count, and — critically — pointers to the actual data blocks on disk (direct pointers plus indirect/double-indirect/triple-indirect pointers for larger files). The filename itself lives only in a directory entry, which maps a name to an inode number — this indirection is exactly why hard links work (multiple directory entries pointing at the same inode).

inode directory entry indirect blocks

6.5 INTERMEDIATE Compare contiguous, linked, and indexed file allocation.

- **Contiguous:** each file occupies a single run of consecutive blocks. Excellent sequential *and* random-access performance (one seek), but suffers external fragmentation and requires knowing the file's size in advance to allocate room for growth.
- **Linked:** each block stores a pointer to the next block of the file, scattered anywhere on disk. No external fragmentation and files can grow freely, but random access is terrible (must traverse the chain from the start), and a single corrupted pointer breaks the rest of the chain.
- **Indexed:** a dedicated index block stores pointers to all of a file's data blocks. Supports efficient random access without contiguous placement, but large files need multiple levels of indirection (as in Unix inodes' indirect/double-indirect/triple-indirect blocks) to keep the index block itself a manageable fixed size.

contiguous allocation

linked allocation

indexed allocation

6.6 INTERMEDIATE How does the OS track free disk space?

Common approaches: a **bitmap/bit vector** with one bit per block (1 = free, 0 = allocated) — compact and fast to scan for contiguous free runs, but requires a full disk scan for some operations and the bitmap itself must be kept in memory for speed on large disks. A **free list (linked list)** chains all free blocks together — no wasted scanning, but poor locality since free blocks end up scattered, and traversal to find several contiguous free blocks is inherently slow. A **grouping** variant improves on the linked list by having each free block store the addresses of several other free blocks (not just the next one), reducing the number of disk accesses needed to locate many free blocks at once.

bitmap

free list

grouping

6.7 INTERMEDIATE

What is journaling, and how does it protect against crash inconsistency?

A multi-block file system update (e.g. creating a file touches the inode, a directory entry, and a free-space bitmap) is not atomic with respect to a crash — if the system loses power midway, the disk can be left in an inconsistent state (e.g. a block marked allocated in one structure but not another). **Journaling** solves this by writing a description of the intended changes to a dedicated append-only log (the journal) *before* applying them to the actual file system structures. If a crash occurs mid-update, on reboot the file system replays the journal to either complete or cleanly roll back the interrupted transaction, rather than requiring a full, slow consistency scan of the entire disk (as the older fsck approach required). Common modes trade durability for performance: **journaling metadata only** (fast, protects structure but not file data) vs **full data journaling** (safer, slower, since actual file content is also logged).

write-ahead log crash consistency fsck

6.8 INTERMEDIATE Explain RAID levels 0, 1, 5, 6, and 10.

LEVEL	TECHNIQUE	FAULT TOLERANCE	USABLE CAPACITY
RAID 0	Striping, no redundancy	None — one disk failure loses everything	100% (N disks)
RAID 1	Mirroring	Survives 1 failure per mirror pair	50%
RAID 5	Striping + single distributed parity	Survives 1 disk failure	(N−1)/N
RAID 6	Striping + double distributed parity	Survives 2 simultaneous failures	(N−2)/N
RAID 10	Mirrored pairs, then striped across pairs	Survives multiple failures (if not same mirror pair)	50%

Interview framing that matters more than the table: RAID 0 buys pure speed at zero safety; RAID 1/10 buy safety and speed by burning capacity; RAID 5/6 buy safety more capacity-efficiently but pay a real write penalty (parity must be recomputed on every write) and a slow, vulnerable rebuild window after a disk failure — which is exactly why RAID 6's second parity block matters at scale: on very large arrays, the rebuild after one failure can take long enough that a second failure during that window is a real risk RAID 5 can't survive.

striping mirroring parity rebuild window

6.9 INTERMEDIATE Compare polling, interrupt-driven I/O, and DMA.

- **Polling:** the CPU repeatedly checks a device's status register in a loop until it's ready, then transfers data itself. Simple, but wastes CPU cycles busy-waiting, especially for slow devices.
- **Interrupt-driven I/O:** the CPU issues a request and moves on to other work; the device raises a hardware interrupt when it's ready, and the CPU's interrupt handler then performs the (still CPU-driven) data transfer. Far better CPU utilization than polling, but still burdens the CPU with the actual word-by-word transfer, and interrupt handling itself has overhead for high-frequency small transfers.
- **DMA (Direct Memory Access):** a separate DMA controller is programmed with a source, destination, and length, and performs the *entire bulk data transfer* directly between the device and memory without CPU involvement, interrupting the CPU only once at completion. Essential for high-throughput devices (disk, network) — the CPU is freed almost entirely from the transfer itself, only setting it up and handling the completion interrupt.

busy-waiting

interrupt handler

DMA controller

6.10 ADVANCED What is a buffer/page cache, and write-through vs write-back?

The OS keeps a **page cache** (or buffer cache) in RAM holding recently read or written disk blocks, so repeated access to the same data avoids a slow physical disk access entirely — this is why a second read of the same file is dramatically faster than the first. **Write-through** caching writes data to both the cache and the underlying disk immediately and synchronously — safer against data loss on a crash, but as slow as writing straight to disk. **Write-back** caching writes only to the cache immediately, marks the block "dirty," and defers the actual disk write to a later, batched point (a periodic flush or when the page is evicted) — much faster, since many writes to the same block get coalesced into one disk write, but risks losing recently-written data that hasn't been flushed yet if the system crashes, which is precisely the failure mode journaling is designed to make recoverable.

page cache

dirty page

write-back vs write-through

6.11

ADVANCED

Hard link vs symbolic (soft) link — what's the difference?

A **hard link** is a second directory entry pointing directly at the same **inode** as the original file — the two names are fully equivalent, indistinguishable at the file-system level, and the underlying data isn't actually removed until every hard link to that inode is deleted (the inode's link count reaches zero). Hard links cannot span file systems/partitions (inode numbers are only unique within one file system) and typically cannot point at directories (to avoid creating cycles in the directory tree). A **symbolic link** is a separate, special file that simply stores a *path string* pointing at another file or directory; the OS transparently follows it on access. Symlinks can cross file systems and point at directories, but become "dangling" (broken) if the target is deleted or moved — a hard link, by contrast, can never dangle, since the inode it references only disappears once nothing links to it anymore.

[hard link](#)[symbolic link](#)[inode link count](#)

6.12 **ADVANCED** Synthesis: what happens, at the OS level, when you run

`cat file.txt` in a shell?

This question exists specifically to check whether you can connect every earlier topic into one causal chain. A defensible answer, in order:

1. The shell **forks** a child process (a near-copy of itself via copy-on-write) and the child calls **exec()** to replace its image with the cat binary.
2. cat issues an `open("file.txt")` **system call**, which traps into kernel mode; the kernel walks the directory structure to resolve the path to an **inode**, checks permissions, and returns a file descriptor.
3. cat calls `read()` in a loop; the kernel first checks the **page/buffer cache** for the requested blocks. On a cache miss, it issues a request to the disk (via the block layer and a **disk scheduling algorithm** if multiple requests are pending), potentially using **DMA** to transfer the data directly into a kernel buffer without CPU involvement, then **interrupts** the CPU on completion.
4. Data is copied from the kernel buffer into the process's own **virtual address space** (translated to physical frames via the process's page table and the **MMU/TLB** on every access), then `write()` to standard output sends it, via another system call, to the terminal driver.
5. Throughout all of this, the **CPU scheduler** is free to context-switch to other ready processes while cat is blocked waiting on disk I/O, and the whole sequence unwinds when cat calls `exit()`, at which point the shell's earlier `wait()` call returns and it reaps cat's exit status before that process can linger as a **zombie**.

end-to-end synthesis

WHAT INTERVIEWERS LISTEN FOR

This exact prompt appears repeatedly in senior/staff-level systems interviews specifically because it can't be memorized — it rewards candidates who've internalized the whole stack rather than six disconnected topics.

- Seek time dominates HDD access cost; SSDs have none of it
- SSTF is greedy/locally fast but can starve; SCAN/C-SCAN trade some speed for fairness
- Inode = metadata + data-block pointers; filename lives only in the directory entry
- Indexed allocation (with indirect blocks) is the practical middle ground on random access + flexible size
- Journaling logs intent before mutating structures, so recovery replays instead of full-disk scanning
- RAID 5 = 1 parity/1 failure; RAID 6 = 2 parity/2 failures, matters most during a slow rebuild window
- DMA offloads the bulk transfer itself, not just the waiting, unlike interrupt-driven I/O

07 Production Engineering & Cross-OS Realities

Textbook OS theory and production engineering diverge in specific, testable ways. This section — sourced from Linux kernel documentation, Microsoft's scheduler internals, and production postmortems published by Cloudflare and Uber — covers the layer where OS theory becomes shipped infrastructure: containers, high-throughput I/O, and cross-platform scheduling decisions.

7.1 INTERMEDIATE

What are Linux namespaces and cgroups, and how do they combine to make a container?

A "container" is not a kernel primitive — Linux has no container system call. It's an application-level illusion built from two independent kernel mechanisms. **Namespaces** control what a process can *see*: each namespace type (PID, mount, network, UTS/hostname, IPC, user, cgroup) gives a process an isolated view of that resource — a process in its own PID namespace believes it's PID 1 and cannot see any process outside it; a process in its own network namespace gets its own private interfaces and routing table. **Cgroups** (control groups) control what a process can *use* — hard and soft limits on CPU time, memory, disk I/O bandwidth, and process count for a group of processes, enforced by the kernel scheduler and memory manager directly.

A container runtime (Docker, containerd, CRI-O) is just automation: on `docker run`, it calls `unshare()/clone()` with namespace flags to isolate the new process's view of the system, writes cgroup limit files under `/sys/fs/cgroup/`, and sets up a filesystem root via `chroot/pivot_root` plus a union filesystem (OverlayFS) for the image layers. This is exactly why containers are dramatically cheaper than VMs — no hypervisor, no guest kernel, no hardware emulation — and also why a container is fundamentally less isolated than a VM: it still shares the host kernel with every other container.

namespaces

cgroups

`unshare()/clone()`

OverlayFS

WHAT INTERVIEWERS LISTEN FOR

"A container is just a regular Linux process with namespaces and cgroups applied" is the sentence senior infra interviewers are fishing for — it demonstrates you know Docker isn't kernel magic.

7.2 ADVANCED

Compare epoll and io_uring for high-performance network I/O — why did Cloudflare and database engineers start moving toward io_uring?

epoll is a *readiness-based* model: a process registers file descriptors of interest, then blocks on `epoll_wait()`, which returns a list of descriptors that are ready for reading/writing — the application still has to issue its own `read()/write()` syscall for each one. At high connection counts and high request rates, this means one syscall to find out what's ready, plus one syscall per actual I/O operation — syscall overhead (each one costs a user/kernel mode transition) becomes the bottleneck.

io_uring is a *completion-based* model built around two ring buffers shared between user space and the kernel: a Submission Queue (SQ) and a Completion Queue (CQ). The application can batch many I/O requests into the SQ and submit them all with a single `io_uring_enter()` call, and later reap many completions from the CQ in one pass — collapsing what used to be dozens of syscalls into one or two. With the right configuration it can even poll the kernel-shared ring memory without issuing a syscall to submit work at all.

The engineering trade-off documented in practice (including Cloudflare's own writing on the AIO/io_submit path that predates and motivated this shift) is nuanced: `io_uring` wins decisively on syscall-heavy, high-fan-out workloads (databases doing many small reads, proxies with huge connection counts), but some highly-tuned `epoll`-based streaming servers still edge it out in raw throughput on certain workloads — the honest interview answer is "it depends on syscall density," not "`io_uring` is strictly faster."

readiness-based vs completion-based

submission/completion queue

syscall batching

REAL INTERVIEW SIGNAL

Infra interviews at network-heavy companies increasingly probe `io_uring` specifically because it's recent enough (mainlined in kernel 5.1) that a candidate's answer reveals whether their systems knowledge stops at a 2015-era textbook.

How does Windows' scheduler differ from Linux's CFS/EEVDF, and how does each platform model threads?

This is a favorite for roles touching cross-platform systems code (game engines, databases, runtimes).

	WINDOWS	LINUX
Scheduler model	Priority-based preemptive multilevel feedback queue; 32 priority levels (0–15 user, 16–31 soft real-time)	Fair-share: CFS (vruntime, red-black tree) → EEVDF since kernel 6.6
Thread representation	Thread is a distinct kernel object (ETHREAD/KTHREAD), independently scheduled within its owning process	No separate thread data structure — a thread is a <code>task_struct</code> that happens to share resources with others; scheduler treats threads and processes identically
Core scaling	Historically tuned for small-to-moderate core counts	Designed to scale load-balancing cleanly across thousands of cores
Customizability	Scheduler internals are fixed, not swappable	Schedulers are modular (e.g. can select CFS, EEVDF, or a real-time policy per-task via <code>sched_setscheduler</code>)

The underlying academic model (ready queue, PCB, context switch, preemption) is identical on both — what differs is engineering philosophy: Windows optimizes around explicit priority classes for desktop responsiveness and real-time-ish work, while Linux optimizes around provable long-term fairness via vruntime accounting.

7.4 ADVANCED

Walk through a real production database deadlock. How does this map to — and diverge from — classic OS deadlock theory?

Concrete case: Transaction A runs `UPDATE orders SET status='shipped' WHERE id=1`, acquiring an exclusive row lock on row 1. Concurrently, Transaction B runs `UPDATE orders SET status='shipped' WHERE id=2`, acquiring an exclusive lock on row 2. Transaction A now tries to update row 2 (blocks, waiting on B's lock); Transaction B tries to update row 1 (blocks, waiting on A's lock). This is a textbook **circular wait** — all four Coffman conditions are present: the row locks are held in a non-shareable (exclusive) mode, each transaction holds one lock while waiting for another, neither lock can be forcibly stolen, and the wait forms a cycle.

Where production databases diverge from the general-purpose OS's usual "ostrich algorithm" stance: a database *cannot* just ignore this, because transactions must eventually commit or fail cleanly — it runs a lightweight, continuous **deadlock detection** algorithm (typically a wait-for graph cycle check, closely related to the OS resource-allocation-graph technique), and upon finding a cycle, picks a **victim** transaction (usually the one that has done the least work, or holds the fewest locks) to abort and roll back, releasing its locks so the survivor can proceed. The application layer is expected to catch the resulting deadlock error and retry the transaction — this retry-on-deadlock pattern is standard production advice specifically because the alternative, avoiding deadlocks entirely via strict global lock ordering across every code path that touches the database, is rarely practical at scale.

wait-for graph

victim selection

row-level locking

retry-on-deadlock

REAL INTERVIEW SIGNAL

SQL/backend interview reports consistently reward candidates who connect this back to the Coffman conditions unprompted — it signals the OS coursework wasn't siloed from how you actually debug production incidents.

7.5 ADVANCED

How do production systems implement adaptive concurrency control, and how does that relate to counting semaphores?

A counting semaphore that caps "at most N threads in this critical section at once" is a fixed, hand-tuned limit. Production services face a harder version of the same problem: the "right" concurrency limit for a downstream dependency isn't static — it changes with backend load, network conditions, and time of day, and a fixed limit set too high causes cascading overload while one set too low wastes capacity. Uber's publicly documented **Cinnamon Auto-Tuner** is a real-world example: rather than a hardcoded semaphore count, it continuously estimates the maximum safe number of concurrent in-flight requests per endpoint by observing live latency signals, and adjusts the effective concurrency limit up or down automatically — conceptually a semaphore whose initial count is recomputed by a feedback controller instead of chosen once by an engineer.

This is the same idea behind TCP congestion control (dynamically discovering the safe send rate) and library-level concurrency limiters (Netflix's `concurrency-limits`, gradient-based AIMD limiters) — the OS-level building block (a bounded counter gating access to a shared resource) is unchanged; what production engineering adds on top is a control loop that keeps re-tuning the bound itself against real-time signal instead of trusting a number chosen at design time.

adaptive concurrency limiting

counting semaphore

feedback control

7.6 INTERMEDIATE

What is the "noisy neighbor" problem, and how do cgroup CPU shares/quotas solve it?

On any multi-tenant host — a Kubernetes node running several pods, or a cloud VM host running several customer VMs — one workload can consume disproportionate CPU, memory, or I/O bandwidth and degrade every other workload sharing that hardware, even though each individual workload's own resource requests looked reasonable in isolation. This is the "noisy neighbor" problem, and it's precisely what cgroups exist to bound. The two relevant controls: **cpu.shares** (now `cpu.weight` in cgroup v2) sets a *relative* priority — under contention, a process with double the shares of another gets roughly double the CPU time, but if the CPU is otherwise idle, a low-share process can still use as much as it wants; **cpu.cfs_quota_us / cpu.max** sets a *hard cap* — a process is throttled and simply cannot exceed X microseconds of CPU time per Y-microsecond period, regardless of idle capacity elsewhere. Kubernetes CPU "requests" map to shares (soft, proportional) and CPU "limits" map to quotas (hard cap) — a direct, production-visible translation of an OS scheduling primitive into a cluster-scheduling concept.

noisy neighbor

cpu.shares / cpu.weight

cfs_quota

K8s requests vs limits

7.7 ADVANCED

What is NUMA, and why does it matter for high-performance production systems?

On multi-socket servers, RAM is physically attached to specific CPU sockets rather than shared uniformly — a core can access memory attached to its own socket ("local") noticeably faster than memory attached to a different socket ("remote"), which it must reach over an inter-socket interconnect. This is **Non-Uniform Memory Access (NUMA)**, and it breaks the simplifying assumption most OS-course memory models make (that all RAM is equidistant from the CPU). A NUMA-unaware scheduler can place a thread on a core in one socket while its allocated memory sits on another, silently costing significant latency on every access. Production-grade schedulers and allocators are NUMA-aware: Linux's scheduler tries to keep a process's memory allocations local to the NUMA node it's running on, and exposes tools (`numactl`, `/sys/devices/system/node/`) for pinning latency-sensitive processes — databases, in-memory caches — to a specific socket's CPUs and memory to avoid cross-socket traffic entirely.

NUMA

local vs remote memory access

numactl

Containers vs virtual machines — where exactly is the isolation boundary in each?

	VIRTUAL MACHINE	CONTAINER
Isolation mechanism	Hypervisor virtualizing hardware; each VM runs its own full kernel	Namespaces + cgroups on a single shared host kernel
Isolation strength	Strong — a kernel exploit in one guest doesn't reach another guest's kernel	Weaker — a host kernel vulnerability can potentially be exploited across containers
Startup time	Seconds to minutes (booting a kernel)	Milliseconds (starting a process)
Density per host	Lower — each guest carries a full kernel/OS memory footprint	Higher — only one kernel to amortize across all containers

The interview-grade nuance: a container is best understood as an *isolation policy applied to an ordinary process*, not a separate execution technology, which is exactly why "container escape" vulnerabilities exist and VM breakout vulnerabilities are comparatively far rarer and more severe when they do occur — the trust boundary sits in a fundamentally different place.

hypervisor isolation

shared kernel

container escape

RAPID-FIRE RECAP — TOPIC 07

- Container = ordinary process + namespaces (visibility) + cgroups (resource limits)
- epoll = readiness + per-op syscalls; io_uring = batched submission/completion rings
- Windows: fixed priority-class MLFQ, threads as first-class kernel objects
- Linux: fair-share CFS/EEVDF, threads are just task_structs sharing resources
- DB deadlocks are detected via wait-for graphs, then resolved by victim rollback + app-level retry
- Adaptive concurrency limiters = a semaphore whose bound is recomputed by a feedback loop
- cpu.shares = soft/relative priority; cpu.quota = hard cap — maps directly to K8s requests/limits
- NUMA: memory locality matters — remote-socket access is measurably slower

Sources & further reading

This guide was compiled from current syllabus references, technical cheat sheets, and aggregated real candidate interview reports, current as of September 2026. Treat any single source's phrasing as illustrative, not authoritative — cross-check against your target company's actual interview format.

[GeeksforGeeks — Top Operating System Interview Questions](#)

[GeeksforGeeks — OS, DBMS & CN Interview Questions](#)

[GeeksforGeeks — Memory Management Interview Questions](#)

[InterviewBit — Operating System Interview Questions](#)

[LeetCode Discuss — Advanced OS FAANG Interview Questions](#)

[Glassdoor — Microsoft interview report: "What is context switching?"](#)

[GeeksforGeeks — Banker's Algorithm in Operating System](#)

[Wikipedia — Bélády's Anomaly](#)

[GeeksforGeeks — Disk Scheduling Algorithms](#)

[TechTarget — Comparing RAID Levels 0, 1, 5, 6, 10, 50](#)

[GeeksforGeeks — Producer-Consumer Problem Using Semaphores](#)

[GeeksforGeeks — Operating System for GATE Exam \(syllabus reference\)](#)

[NGINX Community Blog — What Are Namespaces and cgroups?](#)

[Cloudflare Blog — Missing Manuals: io_uring Worker Pool](#)

[Cloudflare Blog — io_submit: The epoll Alternative](#)

[Uber Engineering — Cinnamon Auto-Tuner: Adaptive Concurrency in the Wild](#)

[Scheduler Behavior in Windows vs Linux](#)

[SQLShack — A SQL Interview Question: Define Deadlocks with an Example](#)

Systems Interview Manual · Operating Systems Edition · Use the "Save as PDF" button (top of page) to export via your browser's print dialog for offline study.